



8.2. Doubly Linked Lists

Previous Sections:

 [8. Linked Lists](#)

 [8.1. Singly Linked Lists](#)

Contents:

[1. Traversal](#)

[2. Searching](#)

[3. Finding the Length](#)

[2. Insertion and Deletion](#)

[2.1. Insertion at the Beginning](#)

[2.2. Insertion at the End](#)

[2.3. Insertion at a Certain Position](#)

[2.4. Deletion at the Beginning](#)

[2.5. Deletion at the End](#)

[2.6. Deletion at a Certain Position](#)

[Summary](#)

[References](#)

In the previous section, we studied **Singly Linked Lists**, where each node stores a reference to only the next node in the sequence. While this structure is simple and memory-efficient, it has one major limitation: we can only move forward through the list.

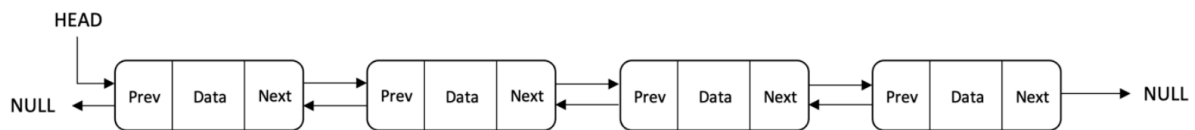
To overcome this limitation, we can use a **Doubly Linked List**.

A Doubly Linked List is a linked list in which each node stores two references:

- A reference to the next node (`next`)
- A reference to the previous node (`prev`)

This allows traversal in both directions, making certain insertion and deletion operations easier and more efficient.

For example:



In this structure:

- Each node knows which node comes before it.
- Each node knows which node comes after it.
- We can traverse both forward and backward through the list.

To implement a Doubly Linked List, we first need a node that stores both references.

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
  
```

Notice that compared to the Singly Linked List node, we now have an additional attribute called `prev`.

The algorithms for **traversal**, **finding the length**, and **searching** are essentially identical to those used in Singly Linked Lists.

Although each node contains both `next` and `prev` references, the following implementations simply traverse from the head toward the end of the list.

1. Traversal

Traversal visits every node in the list.

Implementation:

```
def traverse(self):
    # Empty list
    if self.head is None:
        print("List is empty")
    else:
        temp = self.head

        while temp:
            print(temp.data, end=" <=> ")
            temp = temp.next

        print("None")
```

2. Searching

Searching requires traversing the linked list until the target value is found.

Time Complexity: $O(n)$

Implementation:

```
def search(self, data):
    # Search the value and return its position
    temp = self.head
    position = 1

    while temp:
        if temp.data == data:
            return position

        temp = temp.next
        position += 1
```

```
return -1
```

3. Finding the Length

To determine the size of the list, we count the number of nodes.

Time Complexity: $O(n)$

Implementation:

```
def get_size(self):  
    # Return length of the linked list  
    count = 0  
    temp = self.head  
  
    while temp:  
        count += 1  
        temp = temp.next  
  
    return count
```

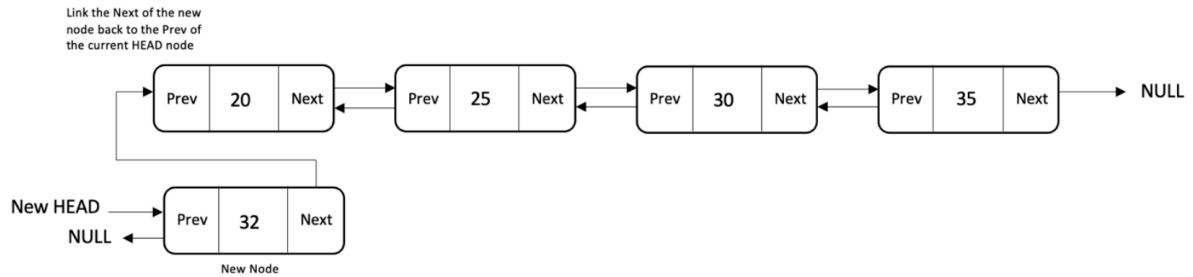
2. Insertion and Deletion

The major advantage of a Doubly Linked List becomes apparent during insertion and deletion operations because every node maintains references in both directions.

2.1. Insertion at the Beginning

To insert a node at the beginning:

1. Create a new node.
2. Point the new node's `next` to the current head.
3. Point the current head's `prev` to the new node.
4. Update the head reference.



Time Complexity: $O(1)$

Implementation:

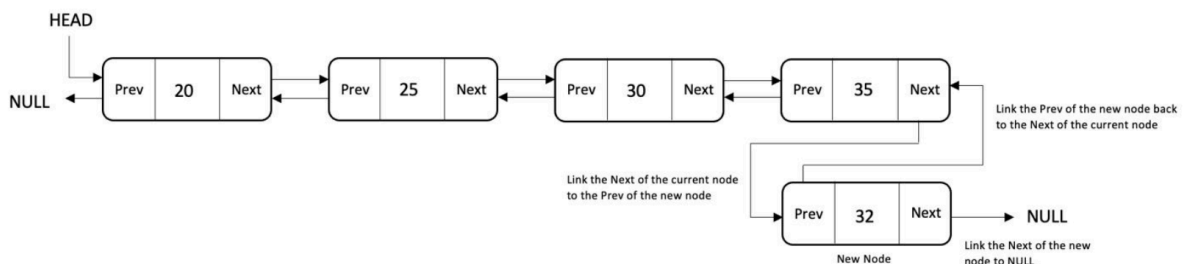
```
def insert_begin(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node
```

2.2. Insertion at the End

To insert at the end:

1. Traverse to the last node.
2. Connect the last node to the new node.
3. Connect the new node back to the last node.



Time Complexity: $O(n)$

Implementation:

```
def insert_end(self, data):
    new_node = Node(data)

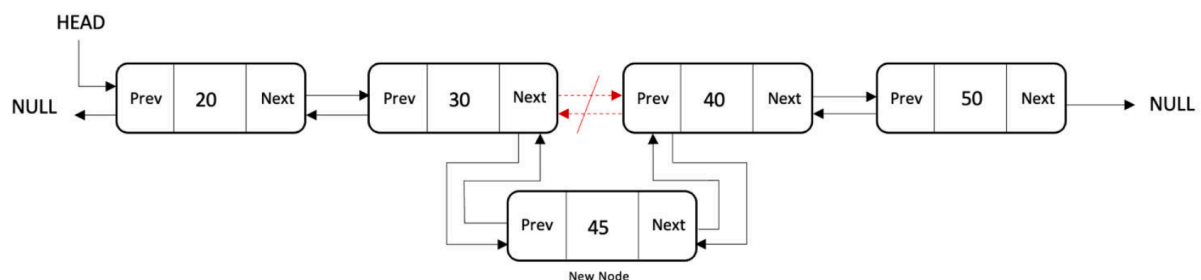
    if self.head is None:
        self.head = new_node
    else:
        temp = self.head

        while temp.next:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp
```

2.3. Insertion at a Certain Position

Insertion in the middle requires updating both the `next` and `prev` references.



Time Complexity: $O(n)$

Implementation:

```
def insert_certain_position(self, data, position):
    if position < 1:
        print("Position is invalid")
        return
```

```

new_node = Node(data)

if position == 1:
    new_node.next = self.head

    if self.head:
        self.head.prev = new_node

    self.head = new_node

else:
    temp = self.head

    for i in range(position - 2):
        if temp is None:
            print("Position is invalid")
            return

        temp = temp.next

    if temp is None or temp.next is None:
        self.insert_end(data)

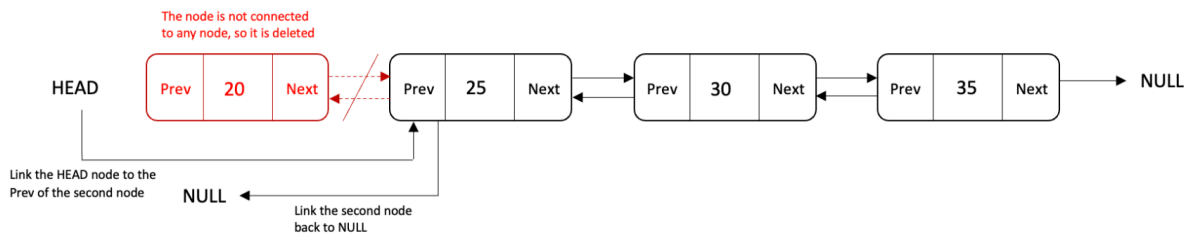
    else:
        new_node.next = temp.next
        new_node.prev = temp

        temp.next.prev = new_node
        temp.next = new_node

```

2.4. Deletion at the Beginning

Deleting the first node involves moving the head to the second node and removing the backward reference.



Time Complexity: $O(1)$

Implementation:

```
def delete_begin(self):
    if self.head is None:
        print("List is empty")
        return

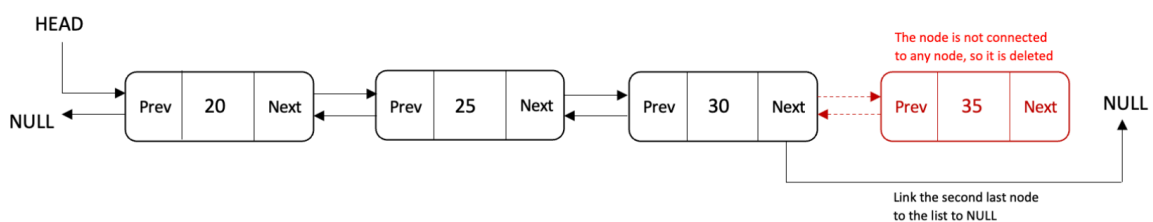
    if self.head.next is None:
        self.head = None

    else:
        self.head = self.head.next
        self.head.prev = None
```

2.5. Deletion at the End

To remove the last node:

1. Locate the last node.
2. Locate the second-last node through the `prev` reference.
3. Set the second-last node's `next` to `None`.



Time Complexity: $O(n)$

Implementation:


```
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next is None:
        self.head = None
        return

    temp = self.head

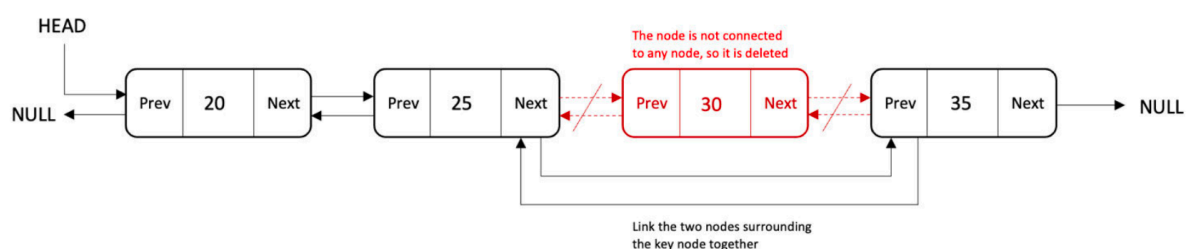
    while temp.next:
        temp = temp.next

    temp.prev.next = None
```

2.6. Deletion at a Certain Position

To remove a node in the middle:

1. Locate the node before the target node.
2. Connect the previous node directly to the next node.
3. Update the backward reference.



Time Complexity: $O(n)$

Implementation:

```
def delete_certain_position(self, position):
    if position < 1:
```

```

        print("Position is invalid")
        return

    if self.head is None:
        print("List is empty")
        return

    if position == 1:
        self.head = self.head.next

        if self.head:
            self.head.prev = None

        return

    temp = self.head

    for i in range(position - 2):
        if temp is None or temp.next is None:
            print("Position is invalid")
            return

        temp = temp.next

    if temp.next is None:
        print("Position is invalid")
        return

    temp.next = temp.next.next

    if temp.next:
        temp.next.prev = temp

```

Below is the full functional Doubly Linked List implementation:

[doubly_linked_lists.py](#)

Summary

In this section, we explored the **Doubly Linked List**, an enhanced version of the Singly Linked List that stores both forward (`next`) and backward (`prev`) references in every node. This additional reference allows traversal in both directions and simplifies many insertion and deletion operations.

We implemented the fundamental operations of traversal, searching, finding the length, insertion, and deletion. Although these operations are similar to those performed on Singly Linked Lists, the presence of the `prev` reference provides greater flexibility when manipulating nodes.

The trade-off is that Doubly Linked Lists require more memory because each node must store an additional reference. However, this extra memory is often worthwhile when bidirectional traversal and efficient node manipulation are important requirements.

Together, Singly Linked Lists and Doubly Linked Lists form the foundation for many advanced linked list structures, including circular linked lists, header linked lists, deques, and numerous dynamic data structures used throughout computer science and software engineering.

References

https://drive.google.com/file/d/1xW6GD_okYUEMMlotBxcIA5RdAe2Rx8Gu/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/doubly-linked-list/>

<https://www.codecademy.com/article/doubly-linked-list>

<https://www.programiz.com/dsa/doubly-linked-list>

Next Section:

 [8.3. Special Operations on Linked Lists](#)

